ME1 Android Port — устройство изнутри (подробно)

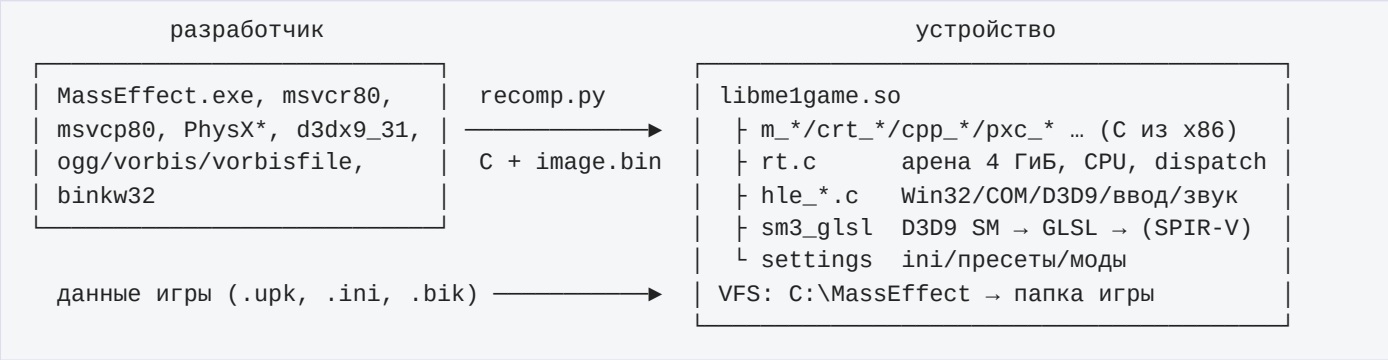
Документ относится к версии 1.0 (рантайм v0.10.13). Он описывает весь путь кода игры: от x86-бинарника до кадра на экране Android-устройства. Краткая версия лежит в `docs/ARCHITECTURE.md`.

---

1. Общая схема

Порт делает с оригинальной PC-версией Mass Effect три вещи:

1. **Статически перекомпилирует** машинный код x86 (EXE и нужные DLL) в C. Перевод делается один раз, на машине разработчика. Получившийся C компилируется под ARM64 и становится частью `libme1game.so`.
2. **Подменяет Windows.** Каждая функция, которую игра импортирует из системных DLL, реализована на C. Так устроены kernel32, user32, gdi32, advapi32, ole32, winmm, Direct3D 9, DirectInput 8, XInput 1.3, OpenAL и другие. Этот слой называется HLE.
3. **Переводит графику и звук.** Вызовы Direct3D 9 и шейдеры SM 1–3 переводятся в OpenGL ES 3 или Vulkan. OpenAL выводит звук в AAudio.



Такой подход отличается от эмулятора (box86/FEX) тем, что во время работы нет ни интерпретации, ни JIT. Весь код игры — это заранее собранный нативный ARM64. Обратная сторона: работает только та версия игры, под которую сделана сборка. Это проверяется по CRC образа.

---

2. Рекомпилятор

2.1 Поиск кода

`tools/recomp.py` разбирает PE (`pefile_min.py`: секции, релокации, импорты, экспорты) и дизассемблирует его через capstone. Где в образе код, а где данные, неизвестно заранее, поэтому код ищется от набора «семян»:

- точка входа и все экспорты;
- указатели на код в данных: любая релокация, которая указывает в исполняемую секцию (vtable, таблицы колбэков, указатели на функции);
- непосредственные операнды `push offset …` и `mov reg, offset …` в уже найденном коде;
- ручные семена (`--seeds`, `ilt_seeds.py`, `tools/seeds/`) — например, цели ILT-переходников;
- точки входа обработчиков исключений из `ehscan.py` (см. 2.4).

`discover()` строит замыкание по целям `call`. `walk()` собирает тело каждой функции. Переход `jmp` в начало другой известной функции считается хвостовым вызовом. Для косвенного `jmp [table+reg*4]` таблица переходов восстанавливается по релокациям. Отдельные эвристики покрывают отрицательную базу и индекс с единицы (так их часто генерирует MSVC).

Для `d3dx9_31` диапазон кода ограничен (`--code-ranges 2b005-21e0a8`), потому что в DLL много данных вперемешку с кодом.

2.2 Модули

Префикс	Модуль	База
`m`	MassEffect.exe	0x10900000 (перебазирован)
`crt`	msvcr80.dll	в области 0x78000000+
`cpp`	msvcp80.dll	в области 0x78000000+
`pxl`	PhysXLoader.dll	своя
`pxc`	PhysXCore.dll	0x12000000
`dx`	d3dx9_31.dll	своя (поставляется в assets APK)
`ogg` / `vb` / `vf`	ogg / vorbis / vorbisfile	0x10100000 / 0x10200000 / 0x10300000
`bk`	binkw32.dll	0x10400000

Всё перекомпилирует `tools/recomp_all.sh`. EXE перебазирован, потому что нижние адреса нужны под стек и кучу. Релокации при этом применяются автоматически.

2.3 Генерация C

Каждая гостевая функция становится C-функцией `void <префикс>_<va>(CPU* c)`.

- **Регистры.** Регистры общего назначения хранятся в `c->eax … c->edi`, `c->esp`. Флаги `cf zf sf of pf` — это локальные переменные C (`FLAGS_DECL`). Компилятор хоста держит их в регистрах и выбрасывает те, что не используются. `pushfd`/`popfd` собирают и разбирают EFLAGS через `MKFLAGS`/`SETFLAGS`.
- **Переходы.** Внутри функции переход — это `goto L_<адрес>`. Условный переход превращается в `if (условие на флагах) goto …`.
- **`call`.** Сначала `push32(c, адрес_возврата)`, потом прямой вызов C-функции, если цель известна. Иначе вызывается `dispatch_call(c, цель)`. Гостевой стек остаётся точной копией стека x86, и код, который читает адрес возврата (`call $+5; pop`), тоже работает.
- **`ret n`** становится `c->esp += 4+n; return;`. Возврат идёт по стеку хоста, а не по адресу из гостевого стека. Поэтому нормальный возврат стоит столько же, сколько возврат из C-функции.
- **Импорты.** `call [IAT]` превращается в `call_import(c, слот)`.
- **`switch`.** Если все цели таблицы лежат внутри функции, генерируется C `switch` с `goto`. Всё остальное уходит в `dispatch_jmp`.
- **LOCK и XCHG с памятью** становятся атомиками хоста (`lk_cas`, `lk_xadd`, `lk_xchg`, `lk_or/and/xor`, seq_cst). Для невыровненных адресов используется глобальный спинлок, для прочих RMW-операций — общий цикл CAS.
- **Особые инструкции.** `int3`, `ud2` и `hlt` вызывают `rt_raise_hw` (аппаратное исключение для SEH). `cpuid` и `rdtsc` обслуживаются хелперами. Неподдержанная инструкция превращается в `UNIMPL(c, адрес, "мнемоника")`: так видно, чего не хватает.

- **x87/SSE/SSE2/MMX** транслирует `tools/fpsse.py`. Стек x87 хранится как `st[8]` в `double` с `ftop`/`fempty`. Отсюда точность 53 бита, что совпадает с режимом MSVC по умолчанию (FCW 0x27F). XMM — 128-битные регистры, MMX хранятся отдельно (не алиасятся на x87).

Пример, упрощённо (настоящий вывод отличается только числом временных переменных):

```
; x86
10a01230  push ebp
10a01231  mov  ebp, esp
10a01233  mov  eax, [ebp+8]
10a01236  test eax, eax
10a01238  je   10a01242
10a0123a  mov  ecx, [eax]
10a0123c  push eax
10a0123d  call dword [ecx+4]
10a01242  pop  ebp
10a01243  ret  4
```

```
void m_10a01230(CPU* c) {
  FLAGS_DECL;
  { uint32_t v = c->ebp; push32(c, v); }
  c->ebp = c->esp;
  c->eax = rd32(c->ebp + 8);                        /* стек: обычная загрузка */
  { uint32_t r = c->eax & c->eax; cf = of = 0;
    zf = (r == 0); sf = r >> 31; pf = PARITY(r); }
  if (zf) goto L_10a01242;
  c->ecx = rdo32(c->eax);                           /* куча: загрузка с порядком */
  { uint32_t v = c->eax; push32(c, v); }
  { uint32_t t = rdo32(c->ecx + 4); push32(c, 0x10a01242u); dispatch_call(c, t); }
L_10a01242:
  c->ebp = pop32(c);
  c->esp += 8; return;                              /* ret 4 */
}
```

2.4 Таблицы исключений

`tools/ehscan.py` разбирает две схемы MSVC:

- **SEH EH3/EH4** — scope table из `_except_handler3/4`. Из неё берутся адреса фильтров и блоков `__except`.
- **C++ EH** — `FuncInfo` с magic 0x19930520–0x19930522. Из него берутся try-блоки, catch-обработчики и продолжения после catch.

На эти адреса нет прямых переходов, поэтому без `ehscan` они остались бы ненайденными. Каждая такая «посадочная точка» становится отдельной C-функцией, а её функция-владелец записывается в таблицу (`owner`). Это нужно рантайму, чтобы прыгнуть в `__except` (раздел 5).

2.5 Выходные файлы

`out/<mod>/` содержит:

- `code_NNN.c` — по 400 функций в файле. Неизменившиеся файлы не перезаписываются, поэтому `make` пересобирает только затронутое;
- `funcs.h` — прототипы;

- `module.c` — таблица `FuncEntry {va, fn, owner}` (отсортирована по va), импорты, экспорты, база, точка входа, `image_size` и `image_crc` (CRC-32 образа);
- `image.bin` — образ модуля после загрузки: секции на своих местах, данные инициализированы.

2.6 Проверка корректности

`tools/difftest.py` генерирует случайные последовательности x86-инструкций и выполняет каждую двумя способами: в Unicorn (эталонный эмулятор) и через recomp → C → gcc. Затем сравнивается состояние: регистры, флаги, x87 (стек, теги, слово состояния), XMM, MMX и затронутая память. Тот же прогон можно выполнить и для ARM64 под qemu-user. Так ловятся ошибки, которые проявляются только на ARM: порядок операций с NaN, сдвиги на ≥ ширину и т. п.

---

3. Ядро рантайма

3.1 Гостевая память

`rt.c` резервирует **арену 4 ГиБ** (`mmap` с `MAP_NORESERVE`), её базу хранит `M`. Гостевой адрес `a` — это хост-адрес `M + a`. Поэтому `rd32(a)` компилируется в одну инструкцию загрузки с базовым регистром. Физическая память выделяется только при первом обращении к странице.

Диапазон	Назначение
0x00100000–0x00400000	стек главного потока (3 МБ под 0x400000)
0x00800000–0x10000000	VirtualAlloc, первая область
0x10000000–0x12400000	образы DLL и EXE (ogg, vorbis, vorbisfile, bink, MassEffect.exe @0x10900000, PhysXCore @0x12000000 …)
0x12400000–0x20000000	VirtualAlloc, вторая область
0x20000000–0x30000000	гостевая куча (HeapAlloc/LocalAlloc/GlobalAlloc, first-fit, `ghalloc`)
0x30000000–0x78000000	VirtualAlloc, третья область
0x78000000+	msvcr80/msvcp80, служебные структуры
0x7FFDE000 / 0x7FFDF000	TEB / PEB главного потока
0xF0000000–0xF0100000	HLE-заглушки (по 16 байт на импорт)

Игра не large-address-aware, поэтому всё её адресное пространство помещается в 2 ГиБ, как и на Windows. VirtualAlloc выдаёт память с гранулярностью 64 КиБ. `MEM_RELEASE`/`MEM_DECOMMIT` возвращают страницы системе через `madvise(MADV_DONTNEED)`.

3.2 Состояние процессора

```
typedef struct CPU {
  uint32_t eax, ecx, edx, ebx, esp, ebp, esi, edi;
  uint32_t fs_base;        /* адрес TEB этого потока */
```

```
  int      df;             /* флаг направления для строковых инструкций */
  double   st[8]; int ftop; uint8_t fempty; uint16_t fcw, fsw;
  uint32_t mxcsr; xmm_t xmm[8]; uint64_t mm[8];
  ...
} CPU;
```

У каждого гостевого потока свой `CPU`, свой гостевой стек и свой TEB. Обращения `fs:[x]` превращаются в `M + c->fs_base + x`.

3.3 Модель памяти: TSO на ARM64

x86 гарантирует сильный порядок (TSO): записи одного ядра видны другим в том же порядке, а загрузки не переставляются друг с другом. ARM64 даёт слабый порядок. Если ничего не делать, lock-free-код игры и движка (очереди рендер-потока, счётчики ссылок, double-checked init) ломается редкими гонками.

Решение — разделить обращения на два класса прямо при генерации:

- **локальные** — `[esp+…]`, `[ebp+…]` в функциях со стандартным фреймом, `fs:[…]`. Другие потоки их почти никогда не видят, поэтому используются обычные `rdN`/`wrN`;
- **все остальные** — `rdoN`/`wroN`, загрузка acquire и запись release. На ARMv8.2 с расширением RCpc acquire-загрузка — это `LDAPR`, она дешевле `LDAR`. Запись — `STLR`. Сборка идёт с `-march=armv8.2-a+rcpc`.

Сборка с `-DRT_NO_TSO` превращает `rdo`/`wro` в обычные обращения. Так можно измерить стоимость барьеров, но игра становится нестабильной.

3.4 Плавающая точка

- x87 эмулируется в `double`. Режим 53 бит совпадает с тем, что выставляет CRT MSVC, а 80-битная точность игре не нужна.
- ARM по умолчанию выдаёт положительный «канонический» NaN, x86 — отрицательный QNaN (0xFFC00000). Там, где это видно коду (сравнение битов, упаковка), NaN приводится к варианту x86.
- `rep stosb/stosd` идут по быстрому пути через `memset`.

3.5 Загрузка модулей

Для каждого модуля загрузчик:

1. берёт `image.bin` либо читает оригинальный PE из `Binaries` (секции раскладываются по RVA, при перебазировании применяются релокации);
2. считает CRC-32 образа и сравнивает с `image_crc` из `module.c`. Если не совпадает, выдаётся ошибка «другая версия игры»: перекомпилированный код соответствует только конкретной сборке;
3. разрешает импорты. Если импорт есть среди экспортов загруженного гостевого модуля, связывается с ним (например, `MassEffect.exe → msvcr80`). Иначе он связывается с HLE-заглушкой. `msvcrt` считается алиасом `msvcr80`. Хук `rt_import_prefer_hle` позволяет предпочесть HLE даже при наличии гостевой реализации.

3.6 Диспетчер вызовов

Прямые вызовы между известными функциями — это просто вызовы C. Через `dispatch_call(c, адрес)` идут косвенные вызовы: виртуальные методы, указатели на функции, `call [IAT]`, если импорт вёл не на заглушку.

```
dispatch_call(c, t):
  если 0xF0000000 <= t < 0xF0100000:  hle_invoke(c, t)          → функция Windows на C
  иначе: fn = dcache[hash(t) & 0xFFFF]                           → кэш с прямым отображением
         при промахе: двоичный поиск t в FuncEntry[] всех модулей
         если t — посадочная точка EH и идёт раскрутка (rt_eh_depth > 0):
               rt_eh_land(...)                                   → раздел 5
         иначе fn(c)
```

Неизвестный адрес — это ошибка «вызов в неперекомпилированный код». Она пишется в лог с адресом, и её закрывают новым семенем рекомпилятора.

`rt_call(c, адрес, аргументы…)` — обратное направление: хост вызывает гостевой код. Так HLE вызывает, например, оконную процедуру, `ThreadProc` или колбэк таймера. На стек кладётся фиктивный адрес возврата `RET_SENTINEL` (0xFFFFFFF0) и вызывается функция.

---

4. HLE: Windows на C

4.1 Механизм заглушек

Каждый HLE-импорт получает 16-байтный слот в области `0xF0000000`. Номер слота — это индекс в таблице `HleDef {dll, имя, nargs, fn, flags}`. `hle_invoke`:

1. по номеру слота находит `HleDef`, увеличивает счётчик вызовов (для статистики), проверяет поток (`hle_thread_check`) и отмечает вход в API для heartbeat;
2. берёт **большой HLE-мьютекс**, если у функции нет флага `HLE_NOLOCK`. Большинство реализаций не потокобезопасны сами по себе, и мьютекс это упрощает. Функции синхронизации и потоков из kernel32 (многие из них блокируют) и таймеры winmm помечены `HLE_NOLOCK`;
3. вызывает C-реализацию. Она читает аргументы макросом `ARG(n)` (это `[esp+4+4n]`) и возвращает значение через `RET(v)` в `eax`;
4. снимает со стека `4 + 4*nargs` байт (stdcall). Для cdecl-функций `nargs = 0`.

4.2 Память и модули

- **`hle_win32.c`** — HeapCreate/HeapAlloc/HeapFree/HeapReAlloc/HeapSize, LocalAlloc, GlobalAlloc. Единый first-fit аллокатор в 0x20000000–0x30000000. Строковые и прочие мелкие функции kernel32.
- **`hle_vmem.c`** — VirtualAlloc/Free/Protect/Query с картой регионов (раздел 3.1).
- **`hle_sys.c`** — дескрипторы модулей, GetModuleHandle, LoadLibrary (возвращает уже загруженный гостевой модуль или псевдомодуль HLE), GetProcAddress (сначала экспорты гостевого модуля, потом таблица HLE), loader lock. `LoadLibrary("openal32.dll")` возвращает псевдомодуль `hle_al_module`.
- **`hle_exe.c`** — остальные системные функции:
  - kernel32: время, информация о системе, окружение, командная строка;
  - user32 — headless: окно существует логически, сообщения формируются из ввода Android;
  - gdi32 — минимум;
  - advapi32 — реестр в памяти с ключами, которые ждёт игра;

- ole32 — CoInitialize, CoCreateInstance для нужных классов;
- winmm — timeGetTime и таймеры;
- dbghelp, wintrust — заглушки;
- dsound сообщает «нет устройства», и игра уходит на OpenAL;
- wsock32 — сеть недоступна.

4.3 Потоки и синхронизация (`hle_thread.c`)

- **CreateThread** создаёт pthread со стеком хоста 16 МиБ: перекомпилированный код глубоко рекурсивен, а каждый гостевой фрейм — это ещё и фрейм C. Потоку выделяются свой `CPU`, гостевой стек нужного размера и TEB (TLS-слоты, указатель на SEH-цепочку, ID потока). Потом через `rt_call` вызывается `ThreadProc`.
- События (manual/auto reset), мьютексы, семафоры и именованные объекты построены на pthread mutex/cond. `WaitForSingleObject` и `WaitForMultipleObjects` поддерживают таймауты и режим wait-all.
- `CRITICAL_SECTION` — это host-мьютекс, привязанный к гостевому адресу структуры. Рекурсивный захват, как в Windows.
- `Interlocked*` — атомики хоста. TLS (`TlsAlloc/Get/SetValue`) хранится в TEB, где его ждёт CRT.
- **Распределение по ядрам (v0.10.13).** Главный поток и рендер-поток UE3 закреплены на больших ядрах SoC, фоновые потоки (стриминг, звук, PhysX) — на малых. Каждому кадру сообщается целевая длительность через ADPF (`APerformanceHint`), чтобы губернатор не занижал частоты. Ручная настройка: `ME1_CPU_PRIME`, `ME1_UCLAMP`.

4.4 Виртуальная файловая система (`hle_fs.c`)

CreateFile, FindFirstFile, GetFileAttributes и прочие функции работают с Windows-путями. VFS переводит их так:

Путь игры	Куда ведёт
`C:\MassEffect\…` (и текущий каталог `Binaries`)	папка игры, выбранная в лаунчере
`C:\Users\Shepard\Documents\…`	папка документов порта (сохранения, `BIO*.ini`, логи, кэши)
прочие `C:\…`	`drive_c` в хранилище приложения

- **Регистр букв.** Windows не различает регистр, Android различает. Поэтому каждый компонент пути ищется без учёта регистра, а содержимое каталогов кэшируется.
- **Моды.** `ME1_MODS` — список каталогов-оверлеев. Файл ищется сначала в модах (выигрывает первый найденный), потом в игре. Каталог `Config` модов открыт только для чтения: запись туда возвращает `ERROR_ACCESS_DENIED`.
- **Подмена ini в памяти.** При открытии `Default*.ini` VFS отдаёт содержимое, в которое `me1cfg_patch_default` уже вписал пользовательские значения из `me1port_overrides.ini`. Сам файл на диске и его время изменения не меняются. Поэтому механизм `[IniVersion]` UE3 не пересоздаёт `BIO*.ini` заново. Ключи `ini` из PhoneModify тоже вписываются в BIOEngine.ini в памяти.

4.5 PhoneModify (`hle_mod.c`)

Включается через `ME1_PHONE_MOD=1`. Поддержаны директивы:

- `maxdraws` / `maxprims` — лимиты отрисовки за кадр (защита слабых GPU);

- `ini` — значения конфигурации;
- `patch` — байтовые патчи по гостевому адресу. Применяются, как только адрес появляется в памяти (например, после загрузки модуля).

4.6 Ввод (`hle_input.c`)

- **DirectInput8** — устройства мыши и клавиатуры. Поддержаны `GetDeviceState` (мгновенное состояние) и `GetDeviceData` (буферизированное кольцо событий). Источник — `hle_input_key` и `hle_input_mouse` из JNI: экранные элементы управления, физическая клавиатура, мышь.
- **XInput 1.3** экспортируется по ординалам (#2 `XInputGetState`, #3 `XInputSetState`). Один виртуальный геймпад наполняется через `hle_xinput_set` данными сенсорного оверлея или реального контроллера. Вибрация считывается обратно и передаётся в Android (`nativePadVibration`). `ME1_XINPUT` управляет поведением.

4.7 Звук (`hle_al.c`)

ISACT, звуковой движок UE3 в ME1, работает через OpenAL 1.1. Порт содержит **собственную программную реализацию OpenAL**:

- одно устройство «Generic Software», до 256 источников;
- буферы в int16: моно и стерео, IMA4 ADPCM, многоканальные форматы (сводятся к стерео);
- pitch с ресэмплингом, модели затухания по расстоянию, конусы направленности, панорамирование;
- EAX и доплер не поддерживаются (игра работает и без них);
- вывод через AAudio (загружается `dlopen`) на «родной» частоте устройства. Если AAudio нет, используется null-sink, и игра идёт без звука. Настройки — переменные `ME1_AL_*`.

Видео (Bink) и музыка (Ogg Vorbis) декодируются перекомпилированными `binkw32` и `vorbis*`.

4.8 Heartbeat и профайлер (`hle_hb.c`)

Отдельный поток раз в `ME1_HB` секунд (по умолчанию 10) пишет строку `[hb]`: состояние каждого гостевого потока (заблокирован в API X или выполняет гостевой код в `fn+off`), счётчики вызовов API, FPS. С частотой `ME1_PROF_HZ` (по умолчанию 100) поток снимает сэмплы: для каждого потока берётся самый внутренний гостевой адрес возврата и сопоставляется с функцией. Это даёт плоский профиль по гостевым функциям без инструментирования кода.

---

5. Исключения (SEH и C++)

MSVC-код активно использует исключения: `__try/__except` в движке и `throw/catch` в C++-части и в msvcp80. В порте вся «логика» исключений — гостевая, выполняется перекомпилированный код CRT. Рантайм предоставляет только примитивы.

- **Гостевые обработчики** `_except_handler3/4` и `__CxxFrameHandler3` из msvcr80 работают без изменений. Они разбирают свои таблицы и вызывают фильтры, деструкторы и catch-блоки.
- **`RaiseException`** (`hle_seh.c`) строит `EXCEPTION_RECORD` и `CONTEXT` в гостевой памяти и обходит цепочку `FS:[0]`, вызывая каждый обработчик. Для вложенных исключений, как в `ExecuteHandler2` из Windows, ставится промежуточный фрейм.
- **`RtlUnwind`** проходит цепочку до целевого фрейма с флагом `EH_UNWINDING`: так выполняются `__finally` и деструкторы.

- **Аппаратные исключения** (`int3`, `ud2`, деление на ноль, обращение к невалидному адресу) пропускаются через тот же механизм как `STATUS_*`. `UnhandledExceptionFilter`, `SetUnhandledExceptionFilter` и `VirtualQuery` (CRT проверяет им стек) тоже реализованы.

Нелокальный переход. В Windows `__except` или продолжение после `catch` получают управление так: ESP/EBP переставляются на фрейм функции-владельца, и выполняется переход внутрь неё. В C так сделать нельзя. Поэтому:

1. Каждая функция, у которой есть посадочные точки, на входе вызывает сгенерированную обёртку над `_setjmp` и кладёт `RtEhFrame` (адрес владельца, гостевой EBP, `jmp_buf`, глубины HLE и EH) в стек фреймов потока.
2. Когда гостевой обработчик решает «прыгнуть» в посадочную точку, `dispatch_call` видит адрес из таблицы посадочных точек при `rt_eh_depth > 0` и вызывает `rt_eh_land`.
3. `rt_eh_land` находит в стеке `RtEhFrame` с тем же владельцем и тем же EBP и делает `longjmp` в него. Стек хоста раскручивается до фрейма владельца.
4. Там посадочная функция вызывается как хвостовой вызов владельца: она продолжает работу с его гостевым фреймом (EBP, локальные переменные) и в итоге делает его `ret`.

Тест `tests/eh` (`mk_eh_test.py` собирает DLL с MSVC-подобным кодом, harness — `runtime/main_eh.c`) проверяет реальную машинерию msvcr80 после рекомпиляции. Он покрывает EH3/EH4-фреймы, фильтры, деление на ноль как аппаратное исключение, `__finally`, глобальную раскрутку, `throw/catch` с деструкторами, `catch(...)` SEH-исключения, rethrow, try/catch внутри catch и параллельные исключения в нескольких потоках.

---

6. Графика

6.1 Объекты Direct3D 9 (`hle_d3d9.c`)

Игра вызывает `Direct3DCreate9`, потом методы COM-интерфейсов. Каждый объект (IDirect3D9, Device, Texture, Surface, VertexBuffer, IndexBuffer, VertexDeclaration, шейдеры, Query, StateBlock) — это **16-байтный гостевой стаб**: `{указатель на vtable, magic, индекс хост-объекта, …}`. Vtable состоит из HLE-заглушек в порядке методов из `d3d9.h`. Вызов `device->SetTexture(…)` из перекомпилированного кода — это косвенный вызов через vtable, и он попадает в `hle_invoke` и далее в C-реализацию.

- `Lock`/`LockRect` возвращают указатель в гостевую память: игра пишет туда напрямую. На `Unlock` данные уходят бэкенду.
- Состояния рендера и сэмплеров, константы шейдеров и привязки кэшируются, а в бэкенд передаются при отрисовке.
- Если нет ни GL, ни Vulkan (тесты, headless-запуски), используется null-бэкенд, и всё, кроме вывода, работает.

6.2 Шейдеры HLSL

UE3 компилирует HLSL во время работы через D3DX. Порт использует **настоящий компилятор**: перекомпилированный `d3dx9_31.dll` (модуль `dx`). Из него берутся `D3DXCompileShader`, ассемблер, препроцессор и дизассемблер. Остальные функции D3DX (математика, загрузка текстур) реализованы через HLE (`ME1_D3DX=hle|all`). Сам DLL лежит в assets APK и при первом запуске копируется в хранилище приложения (`ME1_D3DX_DLL`).

Компиляция медленная (это эмулированный x86-код компилятора), поэтому результат кэшируется:

- ключ — FNV-1a 64 от препроцессированного исходника, имени точки входа, профиля и флагов;
- кэш лежит в `Documents/me1_hlsl_cache`;
- первый запуск компилирует около 130 глобальных шейдеров (≈ 3 минуты), дальше всё берётся из кэша.

6.3 Трансляция байткода (`sm3_glsl.c`)

Байткод D3D9 SM 1.1 / 2.x / 3.0 переводится в GLSL ES 3.00:

- каждый регистр (r, c, v, t, o, a, p) — это `vec4` или `ivec4`/`bvec4`; маски записи и swizzle переносятся напрямую;
- поддержаны управление потоком (`if/else`, `loop`/`rep` со счётчиком `aL`, `break`, `call`/`label`), предикаты и относительная адресация констант (`c[a0.x + n]`);
- константы VS/PS — это uniform-массивы, `def`-константы встраиваются в код;
- входы VS сопоставляются с VertexDeclaration по semantic/usage index, выходы VS — с входами PS;
- **совместимость чисел** (как в DXVK), параметр `ME1_FP_EXACT`:
  - `0` — обычный IEEE;
  - `1` — rcp/rsq/log/exp ограничиваются, как на железе D3D9; `pow` считается как `exp2(y*log2|x|)` без бесконечностей;
  - `2` — дополнительно `0 * x = 0` в mul/mad/dp: без этого шейдеры UE3 дают NaN-пятна и чёрные пиксели.

6.4 Бэкенд OpenGL ES 3 (`hle_d3d9_gl.h`)

GL загружается во время работы (EGL + GLES 3) и не линкуется статически.

- **Ленивое создание.** Текстура создаётся через `glTexStorage` при первом использовании. Глубина-трафарет — renderbuffer D24S8.
- **Буферы.** Статические VB/IB загружаются в GPU на `Unlock`. Динамические и `DrawPrimitiveUP` идут через потоковое кольцо.
- **Программы.** Одна GL-программа на пару VS+PS. Бинарники программ кэшируются на диске (`glProgramBinary`).
- **Порядок строк.** Изображения хранятся в порядке строк D3D (строка 0 — верхняя). Вершинный шейдер переворачивает Y (`dx_fix`), поэтому render target'ы и загруженные текстуры ориентированы одинаково, а переворот нужен только при финальном blit в окно.
- **Потоки.** У каждого потока, который вызывает D3D, свой EGL-контекст в общей группе. Синхронизация идёт через fence и `glWaitSync`.
- **Вывод кадра.** Три слота кадра. Отдельный presenter-поток копирует готовый кадр на поверхность Android с леттербоксингом, так что рендер не ждёт vsync.
- **DXT без S3TC.** Если GPU не поддерживает сжатие S3TC (часть Mali/PowerVR), DXT распаковывается на CPU в RGB5A1/RGBA4.

6.5 Бэкенд Vulkan (`hle_d3d9_vk.h`)

Собственный рендерер на Vulkan 1.0/1.1:

- из расширений обязателен только `VK_KHR_swapchain`. Опциональные возможности (BC-сжатие, анизотропия, линейная фильтрация float-форматов) проверяются и используются, если есть;
- VkImage с собственным суб-аллокатором памяти, глубина D24S8 или D32S8;
- host-visible буферы с переименованием (новый кусок вместо ожидания GPU) и кольцами на кадр;

- **шейдеры:** GLSL из `sm3_glsl` переписывается под Vulkan (layout, descriptor set'ы, push-константы) и компилируется в SPIR-V через `libme1spv.so` (glslang, подгружается `dlopen`). `VkPipelineCache` хранится на диске;
- **пайплайны** кэшируются по ключу (программа, раскладка вершин, состояние рендера, формат render pass), так как D3D9-состояние «динамическое», а Vulkan требует его заранее;
- 3 слота кадров, в полёте не больше 2. Вывод — blit в swapchain или копия через CPU в `ANativeWindow`;
- `ME1_VK_SPLIT=2048` дробит крупные команды: это обход `VK_ERROR_DEVICE_LOST` на части драйверов Mali. `ME1_VK_STRICT` включает строгие проверки для отладки.

6.6 Zink и пользовательские драйверы

Третий вариант — бэкенд GL, работающий на Mesa Zink (GL поверх Vulkan). Mesa 26.2.6 собирается `android/gpu/build_gpu.sh` с патчем порта. `libme1vkshim.so` — минимальный Vulkan-загрузчик на один ICD для Zink. С libadrenotools можно подгрузить пользовательский драйвер Adreno (Turnip и др.) из zip-архива формата GameExDriver (`meta.json`). Лаунчер (`GpuDrivers.java`) импортирует такие архивы и помечает их совместимость.

---

7. Android: процессы и запуск

7.1 Два процесса

- **Лаунчер** (основной процесс): `MainActivity` и экраны настроек. Он находит папку игры (`GameLocator`), читает сведения об устройстве (`DeviceInfo`), управляет драйверами (`GpuDrivers`), раскладкой (`ControlsOverlay`, `ControlsSettings`, `Pad`) и переводом интерфейса (`L`). Настройки читаются и пишутся через `me1port_jni.c`.
- **Игра** (процесс `:game`): `GameActivity` загружает `libme1game.so`. Если игра падает, лаунчер остаётся жив. После выхода игры процесс уничтожается целиком: глобальное состояние Windows-программы нельзя «перезапустить» внутри процесса.

Лаунчер передаёт игре аргументы командной строки, путь к логу и переменные окружения: `ME1_SCREEN`, `ME1_D3DX_DLL`, выбранный рендерер и драйвер, `ME1_MODS`, `ME1_PHONE_MOD` и дополнительные переменные пользователя.

7.2 JNI игры (`game_jni.c`)

Вызов	Назначение
`nativeStart(args, env, log)`	выставляет окружение, перенаправляет stdout/stderr в `Documents/me1exe_run.log`, запускает `me1_exe_main` в потоке игры
`nativeSurface`	поверхность создана, изменена или уничтожена (передаётся рендереру)
`nativeTouch`, `nativeKey`, `nativeMouseRel`, `nativeMouseButton`	ввод → DirectInput / оконные сообщения
`nativePad`, `nativePadVibration`	геймпад → XInput и обратно вибрация
`nativeStatus`, `nativePerf`	статус загрузки и счётчики для оверлея

Код выхода **77** означает, что драйвер GPU падает раз за разом (серия `DEVICE_LOST` или ошибок контекста). Тогда процесс завершается, а `AlarmManager` запускает игру заново.

7.3 Старт игры (`main_exe.c`)

1. Инициализируется арена, TEB/PEB главного потока и HLE.
2. Загружаются модули: `crt`, `cpp`, `pxl`, `pxc`, `m`. Дополнительно, если они есть, — `ogg`/`vb`/`vf`, `bk`, `dx`. Связываются импорты.
3. Вызываются `DllMain` модулей (msvcr80, msvcp80, d3dx9_31…), затем точка входа EXE: старт CRT (куча, TLS, аргументы) → статические конструкторы → `WinMain`.
4. Дальше работает UE3: создаёт потоки, D3D-устройство, читает ini и пакеты. Всё обслуживает HLE.

При сборке с `-DME1_LIB` точка входа называется `me1_exe_main`, и её вызывает JNI. Без этого флага получается исполняемый файл для тестов на Linux/Termux. `run_main.c` (`libme1run.so`) — тестовый запуск без окна: offscreen EGL и сброс кадров в файлы. По `SIGTERM` пишется отчёт (heartbeat, профиль).

---

8. Настройки

Файл	Кто пишет	Что содержит
`Default*.ini` (в игре)	никто	оригиналы, на диске не меняются
`Documents/…/BIO*.ini`	игра	её собственные пользовательские ini
`me1port_overrides.ini`	лаунчер	значения, которые подставляются в Default*.ini при чтении
`me1port.ini`	лаунчер	параметры порта: рендерер, FPS, разрешение, управление…

`settings/me1settings.c` описывает **схему**: каждую настройку, её ini-секцию и ключ, тип, диапазон и пресеты (minimal/low/medium/high/ultra). `ini.c` — парсер и запись ini с сохранением комментариев и порядка.

Автопресет выбирает уровень по:

- классу GPU/SoC;
- RAM: < 3000 МБ → minimal, < 4500 МБ → low, < 6000 МБ → medium;
- частоте CPU: < 1.8 ГГц → не выше low.

FPS выбирается из 30/45/60, разрешение — 16:9, но не больше экрана.

`me1cfg` (`me1cfg_cli.c`) — то же из командной строки: просмотр и изменение настроек, пресеты, `modgen` (генерация мода PhoneModify).

---

9. Производительность

Замеры из `PERF.md`, Adreno 730, тяжёлая сцена:

- игровой поток занят на 46–98 %, ≈ 230 мс на кадр уходят на чисто гостевую работу. Горячих точек нет: время размазано по коду движка;
- главные накладные расходы общие для всего кода: барьеры TSO на каждом обращении к куче, косвенность `fs:`, стек x87 в памяти и вычисление флагов;
- ≈ 4.5 млн вызовов API за 10 с (из них `SetSamplerState` ≈ 1 млн), диспетчер стоит ≈ 0.3–0.5 мкс на вызов.

Уже сделано:

- `LDAPR` (rcpc) вместо `LDAR`;
- `-O2` для EXE;
- dcache для косвенных вызовов;
- распределение потоков по ядрам и ADPF;
- кэши HLSL, программ и пайплайнов.

Возможные следующие шаги: анализ, какие обращения точно локальны (меньше барьеров), кэширование `SetSamplerState` до бэкенда, профилирование по горячим функциям движка.

---

10. Отладка

Средство	Что даёт
`me1exe_run.log`	весь stdout/stderr рантайма и игры
`ME1_HB=N`	строка `[hb]` раз в N секунд: где каждый поток, FPS, счётчики API
`ME1_PROF_HZ=N`	частота сэмплирующего профайлера (функции `fn+off`)
`UNIMPL` в логе	инструкция x86, которую рекомпилятор не поддержал
«вызов в неизвестный адрес»	не найденная функция → добавить семя в `tools/seeds`
`ME1_VK_STRICT`, `ME1_FP_EXACT`	отладка рендера и численных артефактов
`-DRT_NO_TSO`	сборка без барьеров (только для замеров)
`tools/difftest.py`	сверка трансляции инструкций с Unicorn
`runtime/test_*.c`, `tests/`	тесты ФС, D3D9, GL, HLSL→GL, OpenAL, Vorbis, Bink, модов, EH, потоков

---

11. Ограничения

- Поддерживается только та версия файлов игры, под которую собран код (проверка CRC). Для другой версии нужна повторная рекомпиляция.
- Код, который игра сама генерирует или загружает во время работы, не поддерживается: у статической рекомпиляции нет JIT. Для ME1 это не требуется.
- EAX и доплер в звуке отсутствуют. Сеть недоступна.
- 32-битное адресное пространство игры (2 ГиБ, без LAA) ограничивает, сколько ресурсов можно держать загруженными одновременно.
- Производительность упирается в CPU (раздел 9), поэтому на слабых SoC нужны низкие пресеты.